

End-to-End Data Management Pipeline for a Recommendation System (RecoMart)

Course: Data Management for Machine Learning

Assignment: Assignment 1

Submitted By (Group 55 Members):

<i>BITS ID</i>	<i>Name</i>	<i>Email</i>
<i>2025AE05975</i>	<i>Bala Gangadhara Tilak</i>	<i>2025ae05975@wilp.bits-pilani.ac.in</i>
<i>2025AE05064</i>	<i>PADMAWAR SAYALI VENKATESH</i>	<i>2025ae05064@wilp.bits-pilani.ac.in</i>

Submission Date:

02/05/2026

1. Introduction

Modern machine learning systems rely on robust data pipelines to ensure scalability, reproducibility, and reliability. This project implements an end-to-end data management pipeline for RecoMart, an e-commerce platform. The system automates the journey from raw ingestion to a deployable recommendation model using MLOps best practices.

2. Problem Formulation

Business Objective: Improve user conversion rates by 15% through personalized product discovery.

ML Task: Top-K Recommendation (predicting the likelihood of a user interacting with an item).

Success Metrics:

- **Precision@K:** Accuracy of the top recommended items
- **Recall@K:** Ability of the model to retrieve relevant items
- **NDCG@K:** Measures the quality of the ranking order

3. Data Sources & Attributes

The system integrates three primary entities to build the recommendation engine:

Source	Type	Key Attributes
User Interactions	CSV (Batch)	user_id, item_id, timestamp, event_type
Product Metadata	REST API	item_id, category, price, brand, stock_level
User Profiles	Database	user_id, signup_date, location, age_group

4. Data Ingestion & Storage

The ingestion layer utilizes Python scripts with built-in retry logic.

- **Automation:** Designed for periodic batch ingestion
- **Error Handling:** Failures logged with request identifiers
- **Retry Mechanism:** Transient failures are handled using retry attempts with delay (ensures robustness)

Raw Storage (Data Lake Structure):

data/raw/interactions/yyyy-mm-dd/

data/raw/products/yyyy-mm-dd/

5. Data Validation (Quality Gate)

Before processing, data must pass automated validation checks:

- **Schema Validation:** Ensures correct data types and required fields
- **Range Checks:** Validates acceptable value ranges
- **Duplicate Detection:** Identifies redundant records

A data quality report is generated summarizing validation results.

6. Data Preparation & EDA

Outputs:

- Cleaned datasets
- Statistical summaries and visual insights

Preprocessing Steps:

- Missing values handled
- Categorical attributes encoded
- Numerical values normalized

EDA Insights:

- **Sparsity:**

$$1 - \frac{\text{Non Zero Interactions}}{\text{Users} * \text{Items}}$$

- **Long Tail Distribution:**

Majority of interactions concentrated on a small subset of items

These insights justify the use of latent factor models.

7. Feature Engineering & Feature Store

Raw interaction data is transformed into meaningful behavioural features:

- **User Features:** activity frequency, average interaction
- **Item Features:** popularity score, average rating
- **User-Item Features:** interaction strength

SQL Schema for Feature Storage

```
CREATE TABLE user_features (  
  user_id TEXT PRIMARY KEY,  
  user_event_count INTEGER,  
  user_unique_items INTEGER,  
  user_avg_strength REAL,  
  user_total_spend REAL,  
  user_last_event_ts TEXT
```

```

);

CREATE TABLE item_features (
    item_id TEXT PRIMARY KEY,
    item_event_count INTEGER,
    item_unique_users INTEGER,
    item_avg_strength REAL,
    item_purchase_count INTEGER,
    item_avg_price REAL
);

CREATE TABLE user_item_features (
    user_id TEXT,
    item_id TEXT,
    ui_event_count INTEGER,
    ui_max_strength REAL,
    ui_last_event_ts TEXT,
    PRIMARY KEY (user_id, item_id)
);

CREATE TABLE item_cooccurrence (
    item_id TEXT,
    similar_item_id TEXT,
    cooccurrence_score INTEGER,
    PRIMARY KEY (item_id, similar_item_id)
);

```

The feature store enables consistent feature usage across training and inference stages.

8. Data Versioning & Lineage (DVC)

To ensure reproducibility, DVC is used for data versioning.

- Tracks datasets and pipeline stages
- Maintains lineage across transformations
- Links data versions to model training

Key components:

- dvc.yaml → Defines pipeline stages
- .dvc files → Track datasets
- dvc.lock → Captures exact execution state

This ensures that any model can be reproduced using the exact dataset version.

9. Model Training & Experiment Tracking

Algorithm: Singular Value Decomposition (SVD)

Justification:

- Handles sparse interaction data effectively
- Learns latent user preferences
- Enables personalized recommendations

MLflow Tracking

MLflow is used for experiment tracking:

- **Parameters:** model configuration (e.g., number of factors)
- **Metrics:** Precision@K, Recall@K, NDCG@K
- **Artifacts:** trained model and reports

Each experiment is assigned a **unique Run ID**, ensuring traceability between:

- dataset version
 - model parameters
 - evaluation results
-

10. Pipeline Orchestration

The pipeline is structured as a Directed Acyclic Graph (DAG):

Ingestion → Validation → Preparation → Feature Engineering → Training

Each stage depends on the previous stage, ensuring correct execution order.

Logging is implemented across all stages to monitor execution and detect failures.

11. Conclusion & Inference

The pipeline ensures that the RecoMart recommendation system continuously learns from fresh and validated data.

The trained model generates **personalized Top-K recommendations** based on user interaction history. These recommendations improve user engagement and increase conversion rates by presenting relevant products.

The modular design enables scalability and easy integration with production systems.